SVKM'S NMIMS

Deemed-to-be University

MUKESH PATEL SCHOOL OF TECHNOLOGY MANAGEMENT & ENGINEERING

RAILNEXUS

Railway Reservation System

Project Report
Advanced Database Management Systems

Submitted by: Priyanshu Tiwari

Roll Number: [Enter roll number]

Programme / Division: [Enter programme and division]

Submitted to: [Enter faculty name]

Academic Year: 2026–2027

Date: 23 October 2026

Oracle PL/SQL • Redis • MongoDB
React • TypeScript • Express

Abstract

RailNexus is a railway reservation application developed to demonstrate Advanced Database Management Systems concepts through connected user actions. A passenger searches for a train, reserves a ticket, completes a simulated payment, and checks or cancels the reservation. The TTE verifies passengers and records boarding. The Station Master publishes station announcements, while Zone and Super Administrators inspect reports and audit events.

The transactional system uses Oracle Database with a shared reference schema and three zone schemas. PL/SQL packages implement fare calculation, booking, cancellation, payment, announcements and boarding. Keys and constraints maintain referential integrity. A booking-audit trigger creates an outbox event within the transaction, and an announcement trigger records publication activity. Redis supports station caching; MongoDB receives best-effort sensor-reading snapshots.

The demonstration dataset covers 1–30 October 2026. Every regular train class has two usable confirmed seats, one RAC place and five waiting-list places. This intentionally small inventory makes genuine queue transitions visible without hundreds of reservations. Four reservations for the same journey and class produce CFM, CFM, RAC and WL; cancellation promotes the queue in priority order.

This report documents the implemented design, the EER model, relevant source files, transaction behaviour, verified tests and real frontend screenshots. It distinguishes directly demonstrated features from supporting database extensions and records the limitations of the classroom deployment.

Keywords

PL/SQL; railway reservation; EER specialization; transactions; FIFO queue; triggers; distributed schemas; Redis; MongoDB.

Table of Contents

Abstract	2
1. Introduction	4
2. Objectives and Scope	5
3. Requirements and Actors	6
4. System Architecture	7
5. Database Technology Choices	8
6. EER Model — Relationship Overview	9
6.1. EER — Person and Employee Specialization	10
6.2. EER — Core Reservation Relationships	11
6.3. EER — Ticket and Payment Subtypes	12
6.4. EER — Train Specialization	13
7. Logical Schema and Integrity Constraints	14
8. Relational Design and Normalization	15
9. Booking and Availability	16
10. Cancellation and Queue Promotion	17
11. Payment and Refund Processing	18
12. PL/SQL Packages, Procedures and Functions	19
13. Triggers and Automatic Database Actions	20
14. Transactions, Concurrency and Recovery	21
15. Zone Fragmentation and Reporting	22
16. Object-Relational and JSON Extensions	23
17. Sensor Data, Redis and MongoDB	24
18. Authentication and Role-Based Access	25
19.1. Train Search	26
19.2. Limited Seat Availability	27
19.3. Confirmed Reservation and Payment	28
19.4. Queue Before Cancellation	29
19.5. Queue After Cancellation	30
19.6. TTE Boarding	31
19.7. Station Announcement Publication	32
19.8. Passenger Receives the Announcement	33
19.9. Zone Administrator Reports	34
19.10. Super Administrator Audit	35
20. Testing and Results	36
21. Limitations and Future Enhancements	37
22. Conclusion	38
References	39
Appendix A. Concept-to-File Map	40

1. Introduction

A railway reservation system must keep several related records consistent: the passenger, the selected journey, the available seat, the ticket status and the payment. A cancellation also affects other passengers because a queued ticket may become eligible for a berth. These relationships make railway booking a useful setting for demonstrating database design and transaction processing.

RailNexus provides a browser interface for five actor types: Passenger, TTE, Station Master, Zone Admin and Super Admin. Two passenger accounts can remain signed in simultaneously in separate tabs. A change made by one actor becomes visible to another actor after the relevant page or list is refreshed. This makes the database effects observable without executing queries during a presentation.

The project is implemented using a React and TypeScript frontend, an Express and TypeScript API, Oracle PL/SQL business packages, Redis and MongoDB. Docker Compose supplies the database services. The frontend requests data through the API rather than connecting directly to any database.

The core demonstration is a reservation lifecycle. An available berth produces a confirmed ticket. Once both usable seats in the selected class are occupied, the next reservation enters RAC and the following reservation enters the waiting list. Cancelling a confirmed ticket frees the berth and triggers the cancellation procedure to promote the next eligible passenger. The database records the corresponding audit and outbox activity.

Problem addressed

The system must avoid duplicate seat allocation, preserve booking history, maintain the correct queue order and show consistent results across actor interfaces.

2. Objectives and Scope

The primary objective is to demonstrate database concepts through working application behaviour. The project connects everyday operations—booking, cancellation, boarding, announcement publication and reporting—to specific tables, PL/SQL procedures, constraints, triggers and views.

The demonstration period is 1–30 October 2026, inclusive. Six regular passenger services are available daily on their configured routes. Express and Superfast journeys expose the configured sleeper and air-conditioned classes; passenger services use their applicable seating class. Freight services are excluded from passenger search.

Inventory is intentionally limited to two confirmed seats, one RAC place and five waiting-list places per regular journey and coach class. The same train, date and class must be used when demonstrating exhaustion and promotion. Changing the journey or class selects a separate inventory pool.

The implemented scope includes predefined actor accounts, a second passenger account, berth preference, PNR lookup, simulated payments and refunds, station announcements, TTE boarding, zone-scoped reports, cross-zone reporting and event auditing. The interface uses the normal Search, Booking and My bookings pages; there is no separate Live demo section.

Scope boundary

This is an academic prototype. It does not integrate with a railway operator, a bank or a production notification gateway. The three zone fragments are schemas on one Oracle instance, not three physically separate Oracle servers.

3. Requirements and Actors

Functional requirements cover searching, reserving, paying, cancelling and observing the resulting changes in authorized staff interfaces. Different actor types receive different controls and data scope. Authentication uses signed access tokens, while the API validates role membership, record ownership and station or zone assignment.

Non-functional requirements include consistent seat allocation, repeatable demonstration data, traceable changes, readable error messages and independent simultaneous actor sessions. The current frontend stores its access token in sessionStorage so that signing in as a different actor in one tab does not overwrite another tab’s token.

Actor	Primary operations	Visible result
Passenger	Search, reserve, pay, cancel, view Notices	Ticket, seat / berth, queue status and refund
TTE	Select journey, load manifest, mark boarded	Updated boarding status
Station Master	Publish for the assigned station	Announcement appears in Passenger Notices
Zone Admin	Assigned-zone reports and announcements	Zone-level summaries and occupancy
Super Admin	All-zone reports, audit and materialized-view refresh	Cross-zone figures and database event history

Runtime requirements

Node.js 20 or later, npm, Docker Desktop or Docker Engine, a modern browser, and the Oracle, Redis and MongoDB Compose services. Native Windows backend configuration uses the Docker-published Oracle host port; this user’s working port is 1522.

4. System Architecture

The frontend renders role-specific pages and sends HTTP requests to the API. The API authenticates the user, validates input and resolves a public schedule to the owning zone's local inventory. Transactional operations call PL/SQL packages through node-oracledb. Redis and MongoDB provide complementary cache and document-storage functions.

The shared Oracle schema stores reference data and shared features. The North, West and South schemas contain the booking families for their originating-zone journeys. Database links allow shared reporting views to combine zone-local results. The diagram describes the deployed logical architecture, not physically separate database hosts.

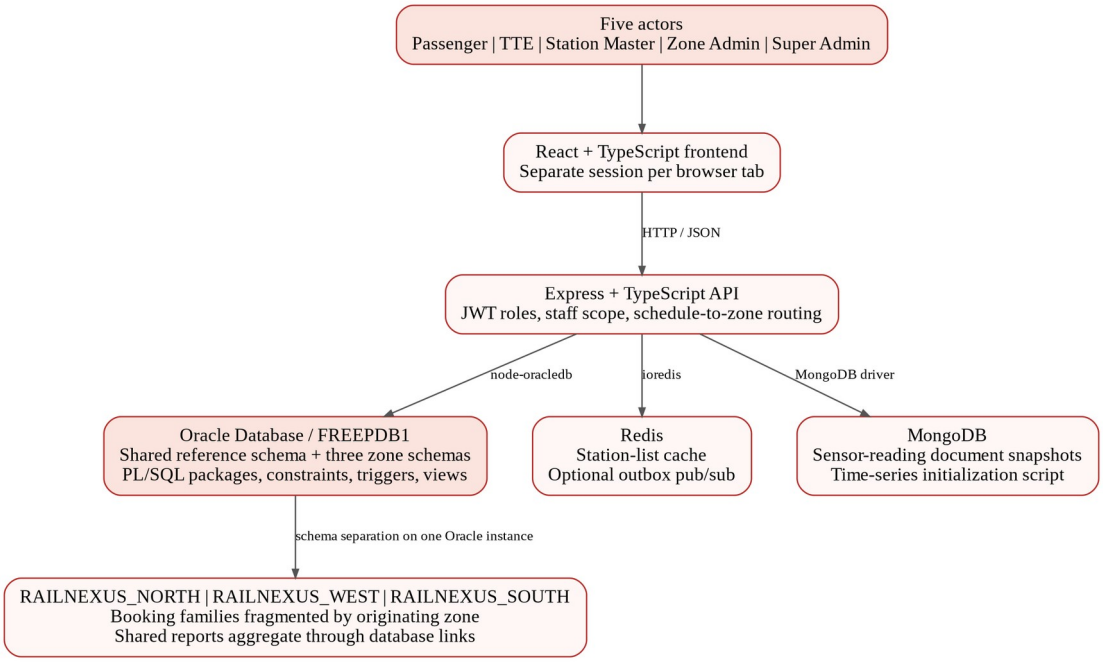


Figure 4.1. Browser, API and three database technologies used by RailNexus.

5. Database Technology Choices

Oracle is the authoritative source for reservations and operational records. Redis and MongoDB are not alternative booking masters: each is used for a separate purpose. The system therefore demonstrates a relational database, a key-value cache and a document-oriented store within one application.

Keeping the booking decision in Oracle means seat allocation, queue changes and audit generation can share a transaction. Cache failures or snapshot-storage failures must not create an alternative reservation result. MongoDB snapshot writes in the IoT path are best-effort; the Oracle sensor ingestion result remains authoritative.

Technology	Data / purpose	Implementation evidence
Oracle Database	Reference entities, schedules, seats, bookings, tickets, payments, refunds, announcements, audit and reports	db/shared_ddl; db/zone_common; db/plsql; db/reporting
Redis	Station-list caching; optional pub/sub from the outbox relay	backend/src/cache; backend/src/outbox/outboxRelay.ts
MongoDB	Sensor-reading document snapshots and time-series initialization	backend/src/iot/iotService.ts; db/mongo/init-timeseries.mjs

Deployment interpretation

One Oracle instance hosts several schemas. Database links and zone routing demonstrate logical fragmentation and federation. The optional outbox relay is disabled by default, so an outbox row alone is not evidence that a downstream consumer received an event.

6. EER Model — Relationship Overview

The overview links reference entities to the booking family. Cardinalities summarize the application design; labels marked “application” describe lifecycle rules that are not all expressed as declarative SQL constraints. Primary and foreign keys are shown inside the entities. Specialization details appear on the following pages.

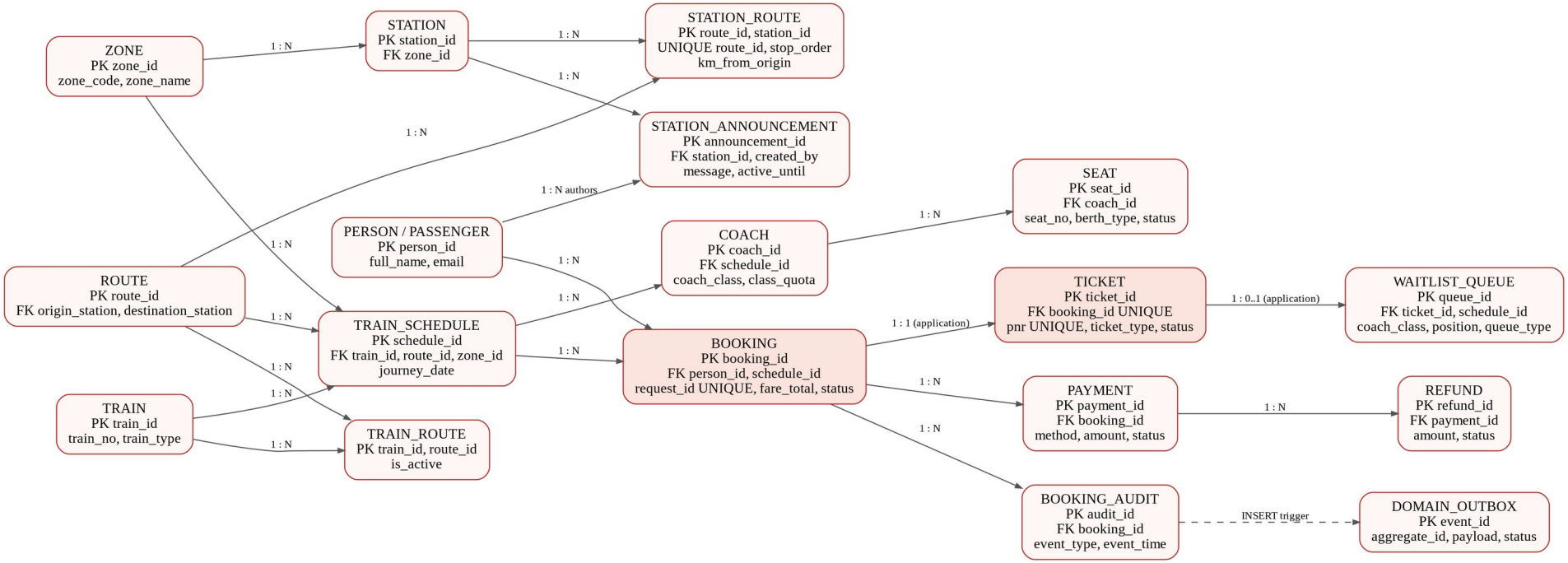


Figure 6.1. Full relationship overview. A standalone SVG and PDF are supplied for zooming.

6.1. EER — Person and Employee Specialization

PERSON contains shared identity and login attributes. PASSENGER and EMPLOYEE reuse person_id as both primary key and foreign key. EMPLOYEE is further specialized into TTE_CONDUCTOR, STATION_MASTER, ZONE_ADMIN and SUPER_ADMIN. This table-per-subtype structure avoids duplicating common identity attributes across role tables.

The “d” circles denote disjoint subtype sets. The person_type discriminator and subtype triggers reject incompatible person categories and conflicting role inserts. The model does not claim that every possible missing subtype row is automatically prevented: creating a supertype without a subtype is not the same as inserting an incompatible subtype.

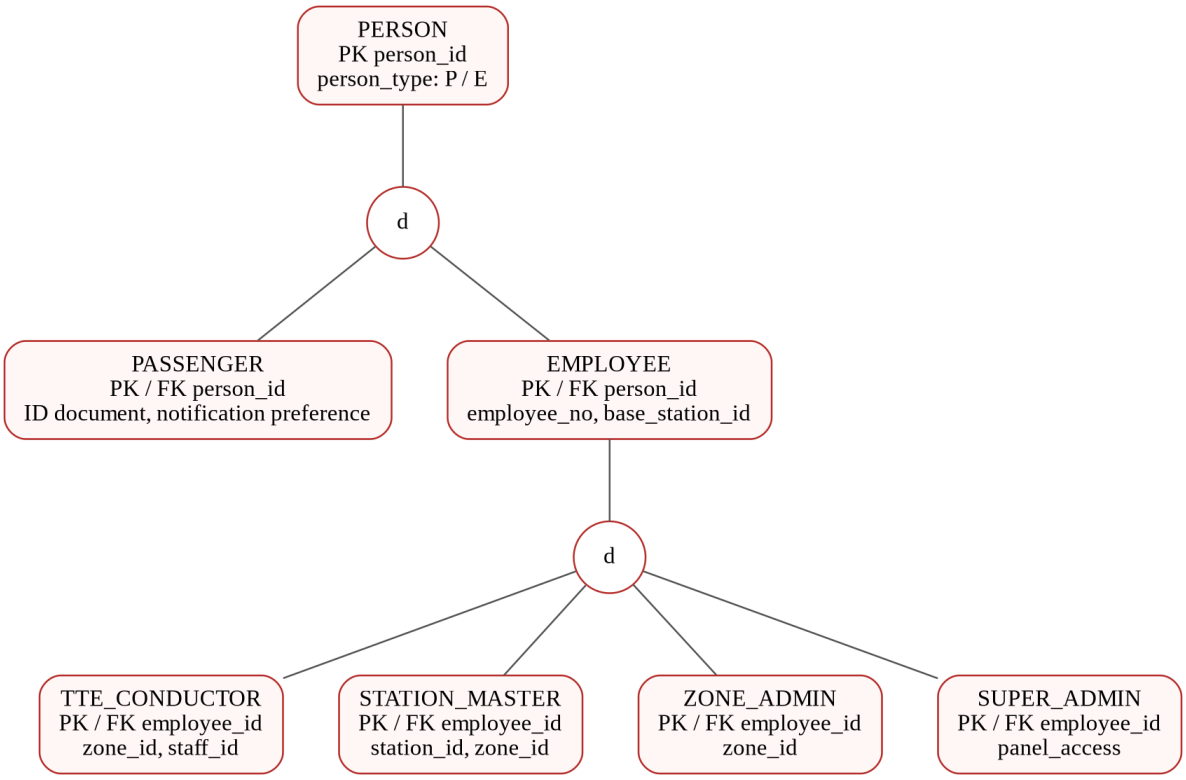


Figure 6.2. Person and employee-role hierarchy; subtype identifiers are inherited PK/FK values.

6.2. EER — Core Reservation Relationships

A passenger can make many bookings, and a scheduled journey can receive many bookings. Each booking produces one ticket through the booking procedure; ticket.booking_id is unique. A schedule has coaches and each coach has seats. A booking can have payment records, while queued tickets have the associated waitlist entries managed by the package.

The queue is scoped by schedule and coach class. Its unique position key prevents duplicate positions within that scope. Changing a date or class therefore changes the relevant queue as well as the available seat inventory.

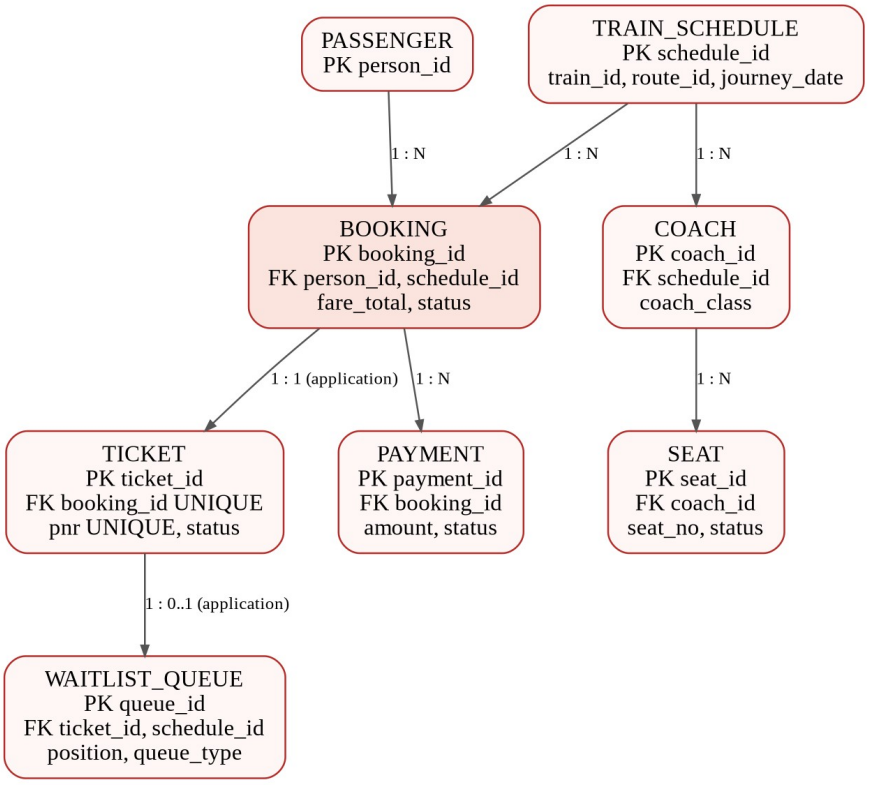


Figure 6.3. Core booking relationships with a readable subset of the full logical model.

6.3. EER — Ticket and Payment Subtypes

TICKET stores the common PNR, booking reference, schedule and status. CONFIRMED_TICKET adds the allocated berth and boarding fields, RAC_TICKET records the application's RAC association, and WAITLISTED_TICKET records waiting-list details. Promotion changes the active subtype rows through pkg_booking.

PAYMENT has UPI_PAYMENT, CARD_PAYMENT and WALLET_PAYMENT detail tables. The method-specific fields are separated from common payment identity, amount and processing state. Payments are simulated in this project. Cancelled tickets remain in the common ticket table with status CNL; the package removes active subtype rows as appropriate.

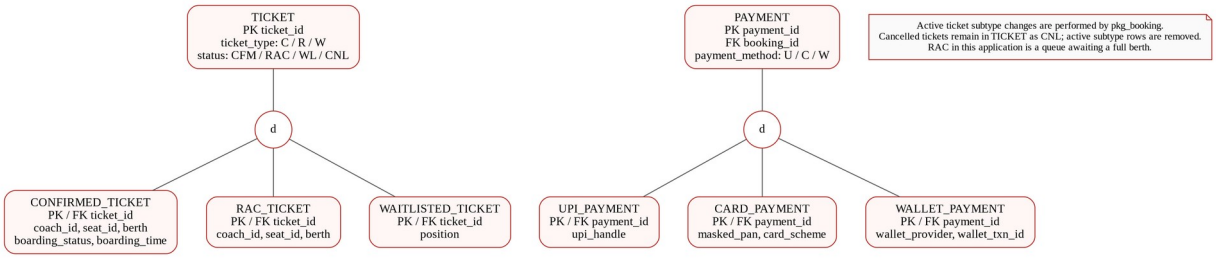


Figure 6.4. Ticket and payment specialization. Active subtype consistency is maintained by package logic.

6.4. EER — Train Specialization

TRAIN is the common service entity and stores train identity, number, name and a type discriminator. EXPRESS_TRAIN, SUPERFAST_TRAIN, PASSENGER_TRAIN and FREIGHT_TRAIN store the subtype-specific attributes using train_id as their inherited PK/FK.

The d circle represents disjoint type sets. Subtype validation triggers check compatible train categories. Passenger search excludes freight services. The fare engine can use Superfast-specific information without repeating common train attributes in every subtype table.

The hierarchy is a conceptual specialization implemented by separate relations. As with the person model, the diagram does not assert a universal database-enforced total-participation guarantee for arbitrary direct SQL inserts.

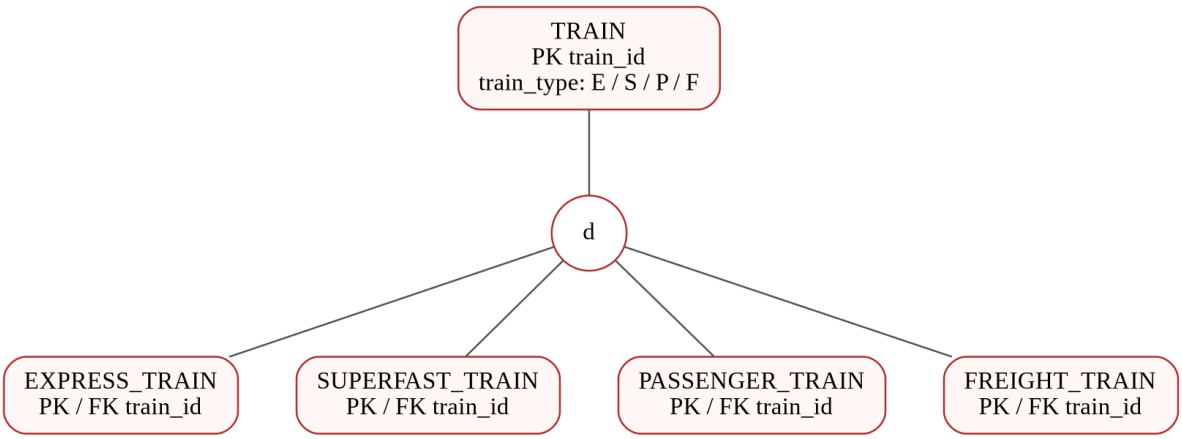


Figure 6.5. Train-type specialization and inherited identifiers.

7. Logical Schema and Integrity Constraints

The booking-family DDL is shared between the catalogue schema and zone schemas. Sequences supply numeric identifiers, and BEFORE INSERT triggers fill identifiers when callers omit them. Foreign keys protect referenced records; unique keys protect PNR values, booking requests and queue positions. CHECK constraints restrict status domains and journey-stop ranges.

The table below summarizes the main relation groups. It is a compact data dictionary, not a replacement for the complete CREATE TABLE definitions. Full definitions are located in db/shared_ddl/01_core_schema.sql and db/zone_common/booking_family.sql.

Relation group	Key attributes	Purpose / integrity
PERSON, PASSENGER, EMPLOYEE	person_id; unique email / phone	Shared identity and subtype references
TRAIN, ROUTE, STATION_ROUTE	train_id; route_id + station_id	Service type, route and ordered stops
TRAIN_SCHEDULE, COACH, SEAT	schedule_id; coach_id; seat_id	Date-specific inventory
BOOKING, TICKET	booking_id; ticket_id; unique request_id / PNR	Reservation and ticket identity
WAITLIST_QUEUE	queue_id; unique schedule + class + position	Ordered RAC / WL queue
PAYMENT, REFUND	payment_id; refund_id; request keys	Capture and refund records
BOOKING_AUDIT, DOMAIN_OUTBOX	audit_id; event_id	Traceable transactional events
STATION_ANNOUNCEMENT	announcement_id; station_id; created_by	Passenger-visible operational message

8. Relational Design and Normalization

The schema separates entities with different identities and update lifecycles. Passenger identity is not repeated in every ticket; the booking stores person_id. Station details are stored once and referenced by station identifiers. A scheduled journey is separate from the train definition because the same train runs on multiple dates.

The route-stop association resolves the ordered relationship between routes and stations. The primary key is route_id with station_id; a unique constraint on route_id and stop_order identifies the ordering within one route. Coach and seat tables similarly separate the coach definition within a schedule from each physical inventory row.

Common person attributes are held in PERSON, while subtype tables hold only category-specific details. Common payment information is held in PAYMENT, while the UPI, Card and Wallet tables hold method-specific information. These decompositions reduce update anomalies and unnecessary null columns.

Not every stored value is intended as a purely normalized derivation. BOOKING retains selected allocation and fare values for the reservation lifecycle, and the object-relational journey layer mirrors relational data for an additional database concept. Reporting views and the materialized view intentionally provide derived representations. The report therefore describes practical normalization choices rather than claiming a formally proved normal form for every relation.

Illustrative functional dependencies

person_id → identity attributes; schedule_id → train, route and date; seat_id → coach and seat number; booking_id → passenger, journey, fare and booking state; ticket_id → booking, PNR and ticket state.

9. Booking and Availability

Passenger search returns only eligible passenger-carrying services for the selected stations and date. The booking page loads valid classes for that schedule, displays remaining confirmed-seat and queue capacity, and accepts an optional berth preference. Berth preference is not a guarantee when the requested berth is unavailable.

pkg_booking.book_ticket locks the schedule, validates the passenger and journey, computes the fare, and attempts allocation in priority order. If a seat is available, the procedure creates a confirmed ticket and occupies its segment. Otherwise, it uses RAC capacity and then waiting-list capacity. If all capacities are exhausted, the procedure rejects the request with an application error.

The request_id uniquely identifies a booking attempt. Repeating the same request does not intentionally create another ticket. The API resolves the shared schedule to the matching owning-zone schedule before the package call, which is important because a catalogue identifier need not equal a local inventory identifier.

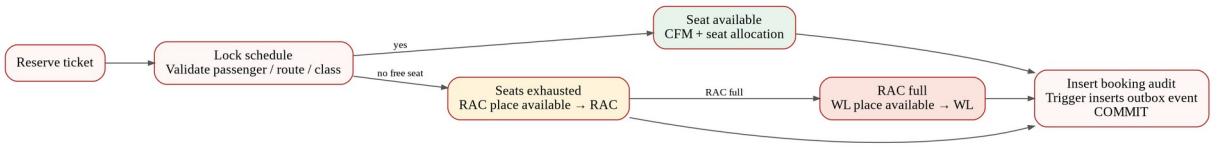


Figure 9.1. Allocation priority: available seat, then RAC, then waiting list.

Relevant files

db/plsql/pkg_booking.sql; backend/src/reservation/reservationService.ts; frontend/src/pages/BookingPage.tsx. Exact current paths are listed in Appendix A.

10. Cancellation and Queue Promotion

Cancellation is performed by pkg_booking.cancel_ticket. The procedure uses the schedule lock, marks the booking and ticket as cancelled, removes active allocation data and releases the occupied segment. It then considers eligible queued tickets for the same schedule and class.

RAC has priority for a freed confirmed berth. The next RAC ticket is promoted to CFM and receives the released allocation. The next WL ticket can then move to RAC when capacity permits. If there is no RAC ticket, an eligible WL ticket can be confirmed directly. The procedure records promotion events as well as the cancellation event.

The small inventory makes the state transitions observable. For a fresh class, four reservations produce two CFM tickets, one RAC ticket and one WL ticket. Cancelling one CFM ticket produces one CNL history entry, two active CFM tickets and one RAC ticket. Free confirmed capacity remains zero because the promoted passenger immediately occupies the freed seat.

Cancellation does not erase history. The normal My bookings page separates Active, All and Cancelled views. A passenger must refresh the list or reopen the ticket to see a promotion made by another session. This is a deliberate frontend refresh workflow rather than an assertion of automatic push delivery.

Stage	Ticket 1	Ticket 2	Ticket 3	Ticket 4
Four reservations	CFM	CFM	RAC	WL
Cancel Ticket 1	CNL	CFM	CFM	RAC
Cancel Ticket 2 next	CNL	CNL	CFM	CFM

Important distinction

Queue promotion is procedure logic. The outbox trigger reacts to the resulting booking-audit inserts; it is not the mechanism that selects the next passenger.

11. Payment and Refund Processing

After reservation, the passenger can choose UPI, Card or Wallet on the payment form. The fare is taken from the server-side booking record. The API validates ownership, method data and amount before the payment package records and captures the simulated payment.

pkg_payment groups payment recording, capture and refund operations. Method-specific tables store the additional UPI, Card or Wallet fields. Request identifiers support repeated-call safety. The classroom flow does not contact a bank or real payment provider.

Cancelling an unpaid reservation releases the inventory but does not create a refund for money never captured. Cancelling a paid reservation requests the refundable captured amount and changes the associated payment state. The response displays the refund amount, while the retained payment and refund rows provide database evidence.

Reservation and payment are separate operations. A reserved confirmed ticket can exist before a simulated payment is captured; this prototype does not implement a production payment timeout that automatically releases an unpaid reservation. That behaviour is recorded as a limitation, rather than hidden by the screenshots.

Relevant files

db/plsql/pkg_payment.sql; db/zone_common/booking_family.sql; backend/src/api/routes/reservations.ts; frontend/src/pages/BookingPage.tsx; frontend/src/pages/PnrPage.tsx.

12. PL/SQL Packages, Procedures and Functions

Packages group related database operations behind a callable interface. Procedures perform workflow changes or return structured results, while functions compute a return value. The API delegates core transactional decisions to these packages rather than reproducing the seat-allocation rules only in the browser.

pkg_fare.compute_fare chooses an effective fare rule, multiplies the route distance by the rate, applies the configured minimum and maximum, and includes the Superfast surcharge when applicable. pkg_tte.board_ticket validates a confirmed ticket and records boarding. pkg_announcements.publish validates the station and actor, inserts the announcement and commits it.

Package / routine	Concept	Observed application effect
pkg_booking.book_ticket	Procedure, locking, exception handling	Seat allocation or RAC / WL reservation
pkg_booking.cancel_ticket	Procedure, transaction, ordered selection	Cancellation and queue promotion
pkg_fare.compute_fare	Function and effective-date query	Server-computed booking fare
pkg_payment	Procedures and subtype persistence	Simulated payment / refund
pkg_tte.board_ticket	Procedure and row update	TTE boarding changes to Yes
pkg_announcements.publish	Procedure and database authorization	Message appears in Passenger Notices
pkg_reporting	Procedure for report refresh	Refreshes reporting materialized view
pkg_iot	Procedure-based sensor processing	Reading classification and alert record

13. Triggers and Automatic Database Actions

Triggers execute in response to database changes. In RailNexus they support generated identifiers, subtype validation and event auditing. Their effect is distinct from the explicit booking and cancellation procedure calls.

When a package inserts a BOOKING_AUDIT row, trg_audit_to_outbox inserts a DOMAIN_OUTBOX event in the same local transaction. If the transaction rolls back, the associated audit and outbox insert roll back together. The package creates the event-specific audit record; the trigger bridges that record into the outbox.

The announcement audit trigger runs after insertion or update of STATION_ANNOUNCEMENT and adds an ANNOUNCEMENT_AUDIT record. Subtype validation triggers check discriminators and sibling membership when relevant rows are inserted or updated. Sequence-based identifier triggers populate missing primary-key values.

Trigger / group	Source file	Purpose
trg_audit_to_outbox	db/plsql/outbox_audit_trigger.sql	Booking audit → durable outbox event
trg_announcement_audit	db/plsql/pkg_announcements.sql	Announcement insert / update auditing
trg_passenger_disjoint; trg_employee_disjoint	db/shared_ddl/ 01_core_schema.sql	Person-category compatibility
Employee-role / train-subtype triggers	db/shared_ddl/ 01_core_schema.sql	Reject conflicting subtype membership
BEFORE INSERT identifier triggers	Core schema and booking-family DDL	Fill absent IDs from sequences

What the audit proves

An audit or outbox row proves that its database action occurred. It does not, by itself, prove that a message was delivered to Redis, MongoDB, a phone or an external service.

14. Transactions, Concurrency and Recovery

Atomicity is important because a reservation changes multiple rows. Booking identity, ticket state, seat allocation, queue position and audit activity must agree. The package commits a successful operation and rolls back on an exception, avoiding a partially completed booking decision.

Booking and cancellation acquire the schedule lock. Candidate seat selection also uses SELECT FOR UPDATE with SKIP LOCKED. These mechanisms coordinate conflicting operations and prevent two concurrent requests from simply selecting the same available row. The design favors correctness and a simple classroom demonstration over maximum booking throughput.

Consistency is supported by foreign keys, unique request identifiers, unique PNR values, status checks and the package's workflow rules. Isolation is achieved through Oracle's transactional behaviour plus explicit row locks where allocation decisions conflict. Durability comes from committed Oracle records and retained audit/outbox rows.

The project does not claim an atomic transaction spanning Oracle, Redis and MongoDB. Oracle remains the reservation authority. Cache updates and document snapshots are supporting operations, and asynchronous propagation depends on configuration. This boundary avoids incorrectly describing the application as a cross-product two-phase commit system.

Demonstrated evidence

The real API regression checks ownership, request replay, correct promotion, refunds and seat consistency. The browser test verifies the same booking and cancellation transitions through the ordinary pages.

15. Zone Fragmentation and Reporting

The booking family is deployed in RAILNEXUS_NORTH, RAILNEXUS_WEST and RAILNEXUS_SOUTH. A journey belongs to the zone of its route’s origin. The API uses zone routing to send reservation operations to the owning schema. Shared person, train, station and fare reference data is made available to those schemas through the configured reference access.

The shared reporting layer combines zone data through database links. Views such as vw_live_bookings, vw_zone_summary and vw_occupancy_by_schedule expose operational aggregates. Filtering by the authorized zone produces the Zone Admin view; Super Admin can inspect the combined result.

Views reflect current underlying rows when queried. The revenue-by-train materialized view stores a derived result and is refreshed on demand by an authorized Super Admin. The interface therefore includes a materialized-view refresh control alongside normal report reload controls.

The deployment uses one Oracle instance with multiple schemas, which demonstrates logical horizontal fragmentation and federated queries. It is not evidence of three independent physical database nodes, replicated failover or a measured distributed-performance improvement.

Relevant files

db/distributed/db_links.sql; db/distributed/distributed_views.sql; db/reporting/report_views.sql; db/plsql/pkg_reporting.sql; backend/src/db/zoneRouter.ts; backend/src/reporting/reportingService.ts.

16. Object-Relational and JSON Extensions

The object-relational layer represents a route journey as route_journey_t, containing a nested collection of segment_t objects. A member function computes total distance from the collection. pkg_journey provides refresh and traversal operations over this derived representation of relational route data.

The JSON layer supplies flexible rule attributes and parsing helpers. JSON columns and rule documents support semi-structured extensions without requiring every optional property to become a new fixed column. pkg_json_rules includes helper functions for reading rule values and computing rule-related amounts.

These are supporting ADBMS implementations in the repository. The primary frontend reservation path and the screenshot evidence focus on ordinary bookings, payments, queue promotion, announcements, boarding and reports. The existence of an object or JSON script is not presented as proof that every optional extension is executed by each normal booking.

In design terms, the object layer provides a derived structured representation and JSON provides flexible attributes. Neither replaces the normalized booking tables as the authoritative reservation record.

Concept	Implementation files	Role
Object types and member function	db/or/route_journey_types.sql	segment_t, segment_tab_t, route_journey_t
Object storage and traversal	db/or/route_journey_tab.sql; db/or/pkg_journey.sql	Derived journey representation
JSON migration and rule helpers	db/json/ migrate_json_columns.sql; db/json/pkg_json_rules.sql	Semi-structured attributes and parsing

17. Sensor Data, Redis and MongoDB

The staff alert interface provides a sensor selector and a reading input. Sensor ingestion calls the Oracle IoT package in the owning zone. Threshold processing classifies the reading and creates the corresponding alert when needed. Authorized staff can acknowledge an alert through the interface.

After Oracle processing, the API attempts a MongoDB sensor snapshot. The time-series initialization script defines a collection using a timestamp field and sensor metadata, with an index for sensor/time access. Snapshot storage is best-effort and does not replace Oracle's transactional sensor result.

Redis is used for station-list caching, reducing repeated reference-data reads when the configured cache is available. The optional outbox relay also has a Redis publication path. Because relayEnabled defaults to false, this report does not claim automatic event delivery merely from the presence of an outbox record.

These functions illustrate the different data models: strongly constrained relational transactions in Oracle, fast key-value caching in Redis, and flexible time-oriented sensor documents in MongoDB. The three technologies serve different responsibilities within the same application.

Relevant files

db/plsql/pkg_iot.sql; db/iot/seed_sensors.sql; backend/src/iot/iotService.ts; db/mongo/init-timeseries.mjs; backend/src/cache; backend/src/outbox/outboxRelay.ts.

18. Authentication and Role-Based Access

The API issues a signed access token after checking an enabled user account. Requests carry that token in the Authorization header. requireAuth verifies the token; requireRole restricts the route to the allowed actor types. Staff-scope checks additionally restrict station and zone access.

Passenger booking lookups and cancellations enforce ownership. A passenger cannot publish announcements or use administrator reporting routes. A Station Master can publish for the assigned station, and the PL/SQL announcement package independently validates the actor’s authorization. Zone Admin reports are filtered to the assigned zone.

The frontend stores the token per browser tab using sessionStorage. This fixes the situation where a Station Master page remained visible while another tab overwrote a shared token with a Passenger login. The displayed role and the request token can now remain aligned in simultaneous actor demonstrations.

Predefined demonstration accounts simplify classroom use; they are not a production identity-management system. Access tokens have a limited lifetime, so signing in again is appropriate after expiry. The report omits passwords, tokens and database connection secrets from screenshots and diagrams.

Implementation evidence

backend/src/api/middleware/auth.ts; backend/src/auth/service.ts; backend/src/auth/tokens.ts; backend/src/api/routes/operations.ts; frontend/src/auth/AuthContext.tsx; frontend/src/lib/api.ts.

19.1. Train Search

The passenger selects the origin, destination and a journey date within 1–30 October 2026. Search returns eligible scheduled services. The screenshot is taken from the running application, not a design mock-up.

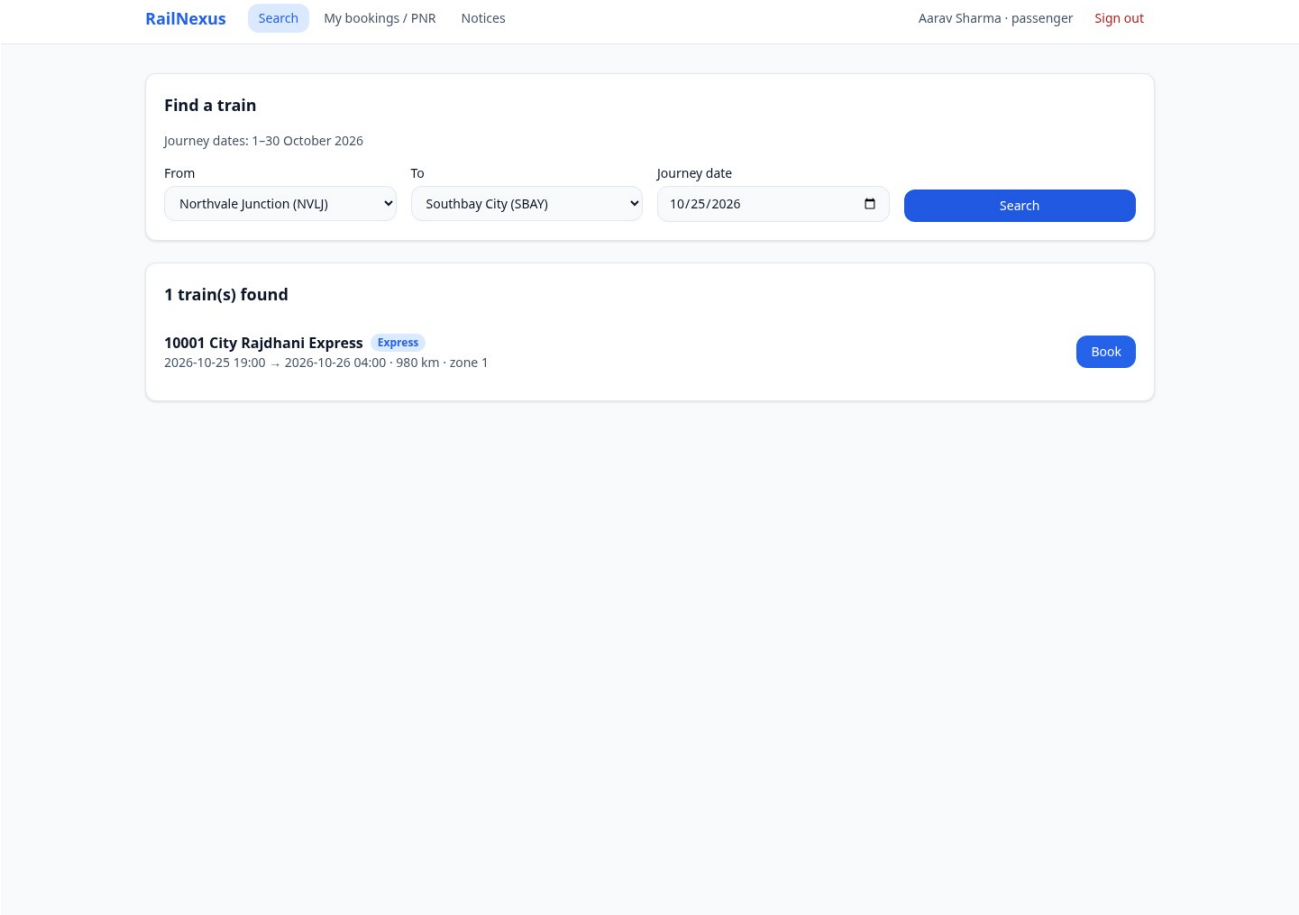


Figure 19.1. Actual RailNexus frontend screenshot, captured 23 October 2026.

19.2. Limited Seat Availability

The ordinary Booking page shows two confirmed seats available in SL, one remaining RAC place and five remaining waiting-list places. These queue numbers are remaining capacities, not counts of people already waiting.

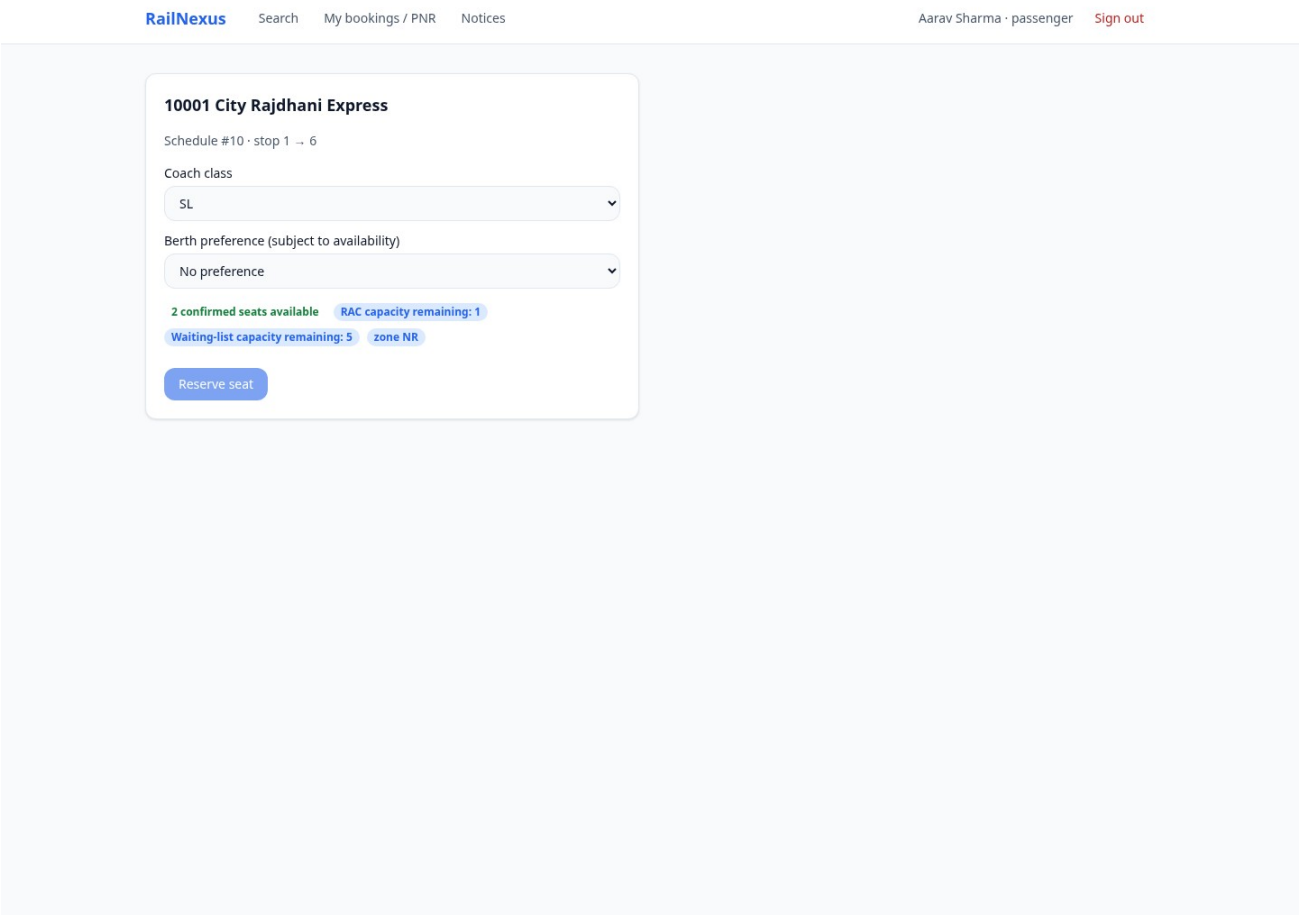


Figure 19.2. Actual RailNexus frontend screenshot, captured 23 October 2026.

19.3. Confirmed Reservation and Payment

A reservation produces a real PNR and confirmed ticket. The same page displays the allocated coach/seat and the result of simulated payment capture. Payment processing is separate from reservation creation.

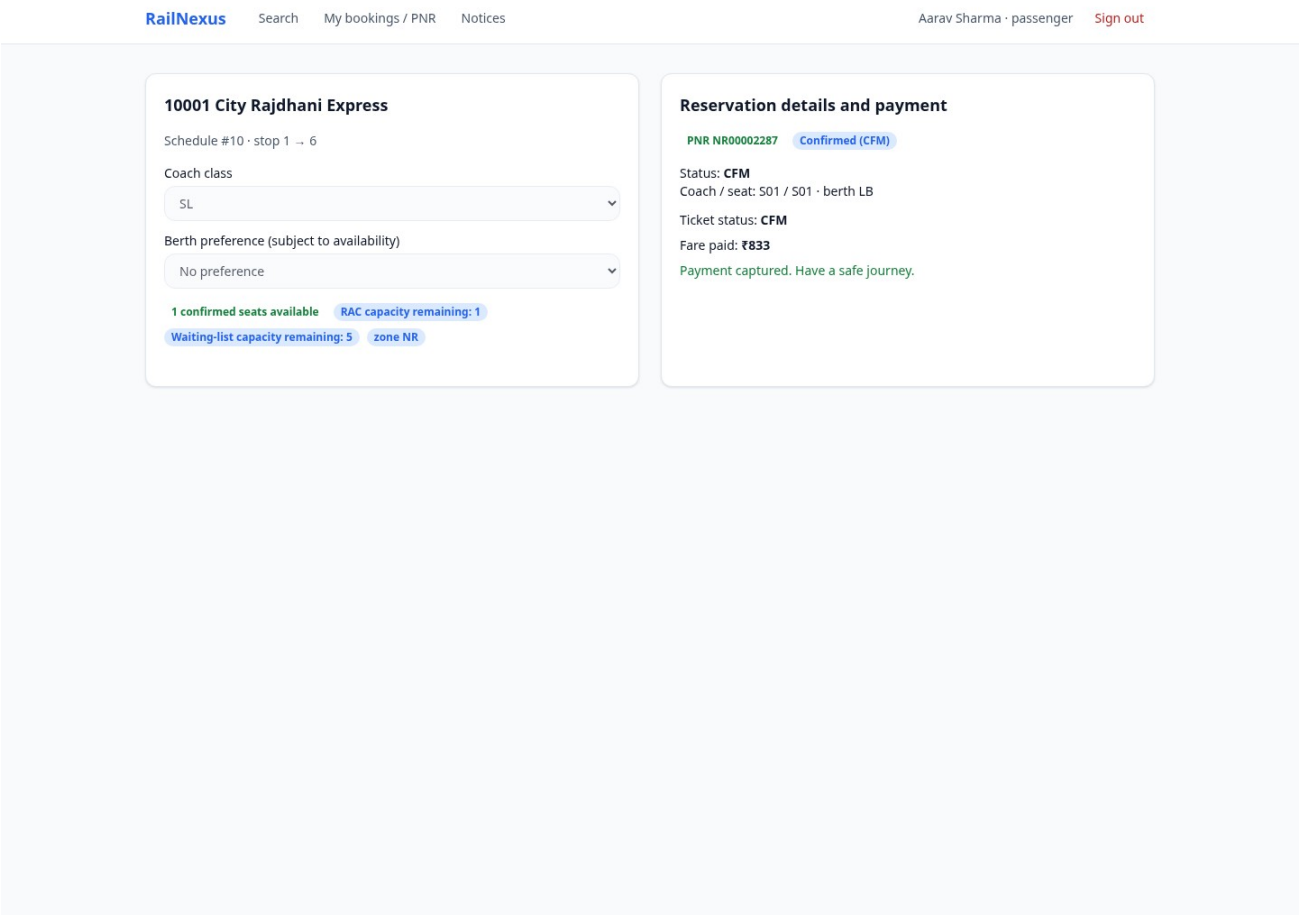


Figure 19.3. Actual RailNexus frontend screenshot, captured 23 October 2026.

19.4. Queue Before Cancellation

Meera’s active bookings show RAC and Waiting list after Aarav occupies both confirmed seats for the same train, date and class. Two separate passenger sessions were used to capture this state.

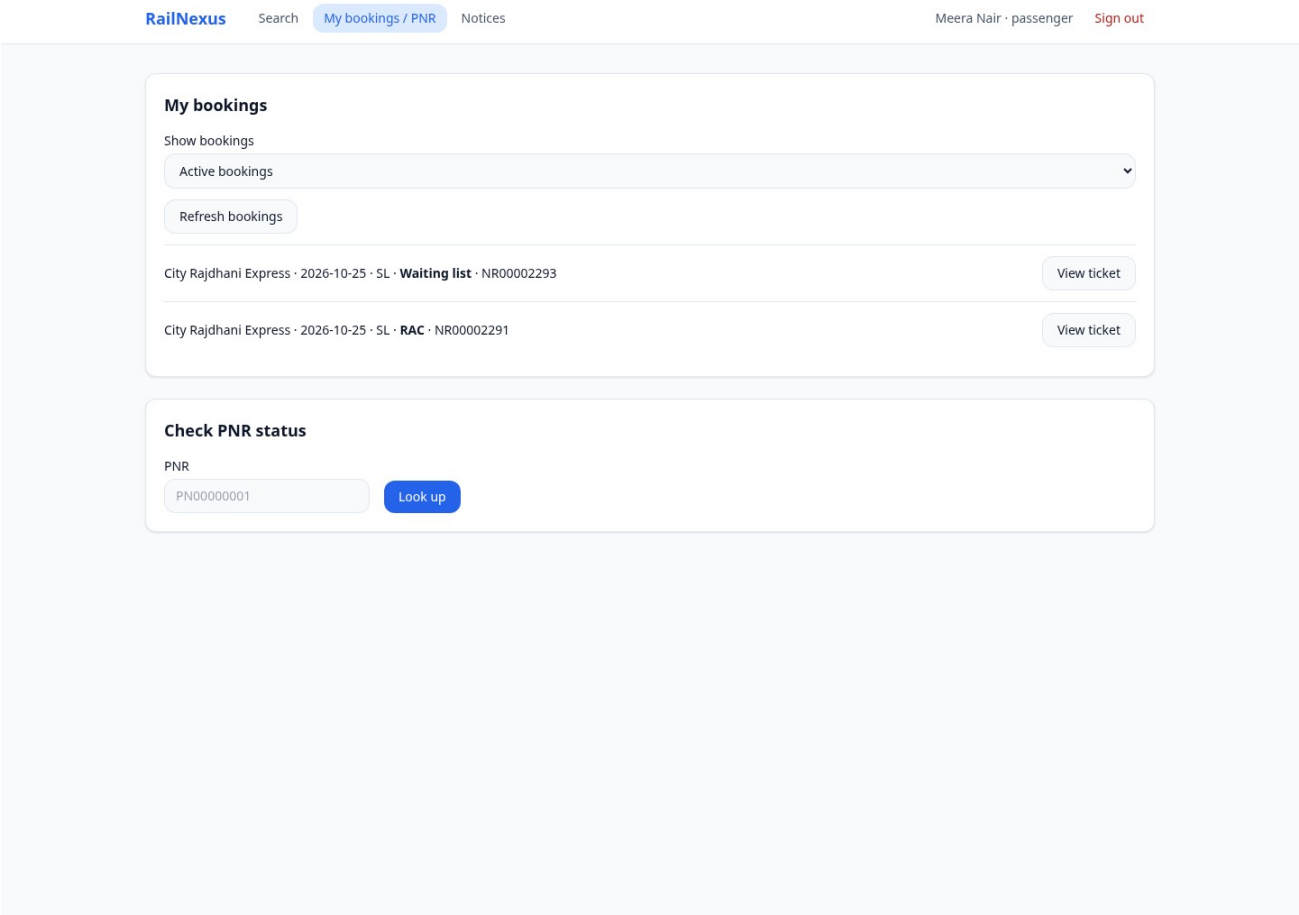


Figure 19.4. Actual RailNexus frontend screenshot, captured 23 October 2026.

19.5. Queue After Cancellation

After Aarav cancels a confirmed reservation and Meera refreshes My bookings, Meera’s RAC ticket becomes Confirmed and her waiting-list ticket becomes RAC. The screenshot records the resulting state, not a manually changed label.

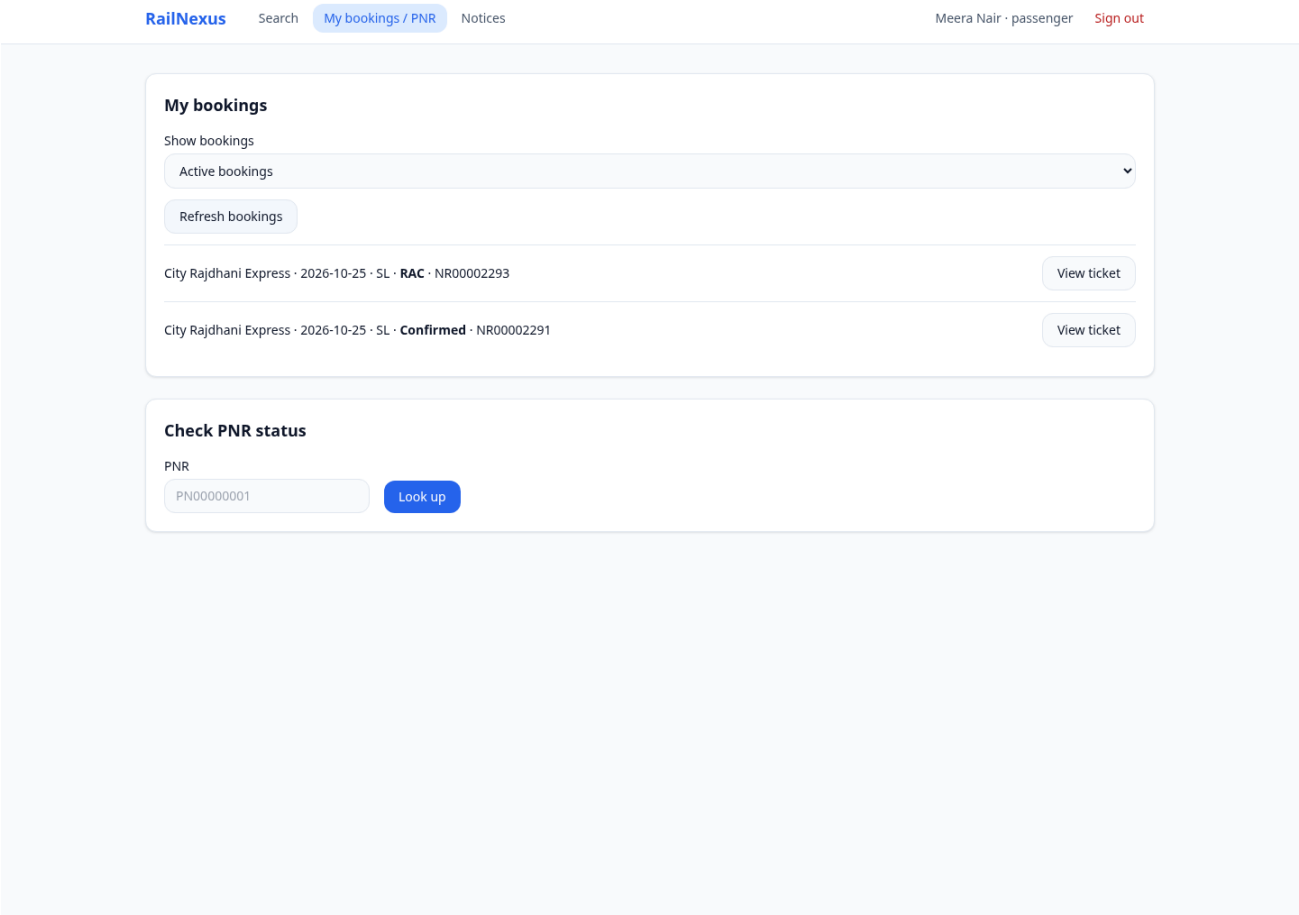


Figure 19.5. Actual RailNexus frontend screenshot, captured 23 October 2026.

19.6. TTE Boarding

The TTE selects the same journey, loads its passenger list and marks the promoted confirmed passenger as boarded. The row changes to Boarded: Yes. The TTE manifest lists active CFM / RAC tickets; cancelled tickets are excluded from that active list.

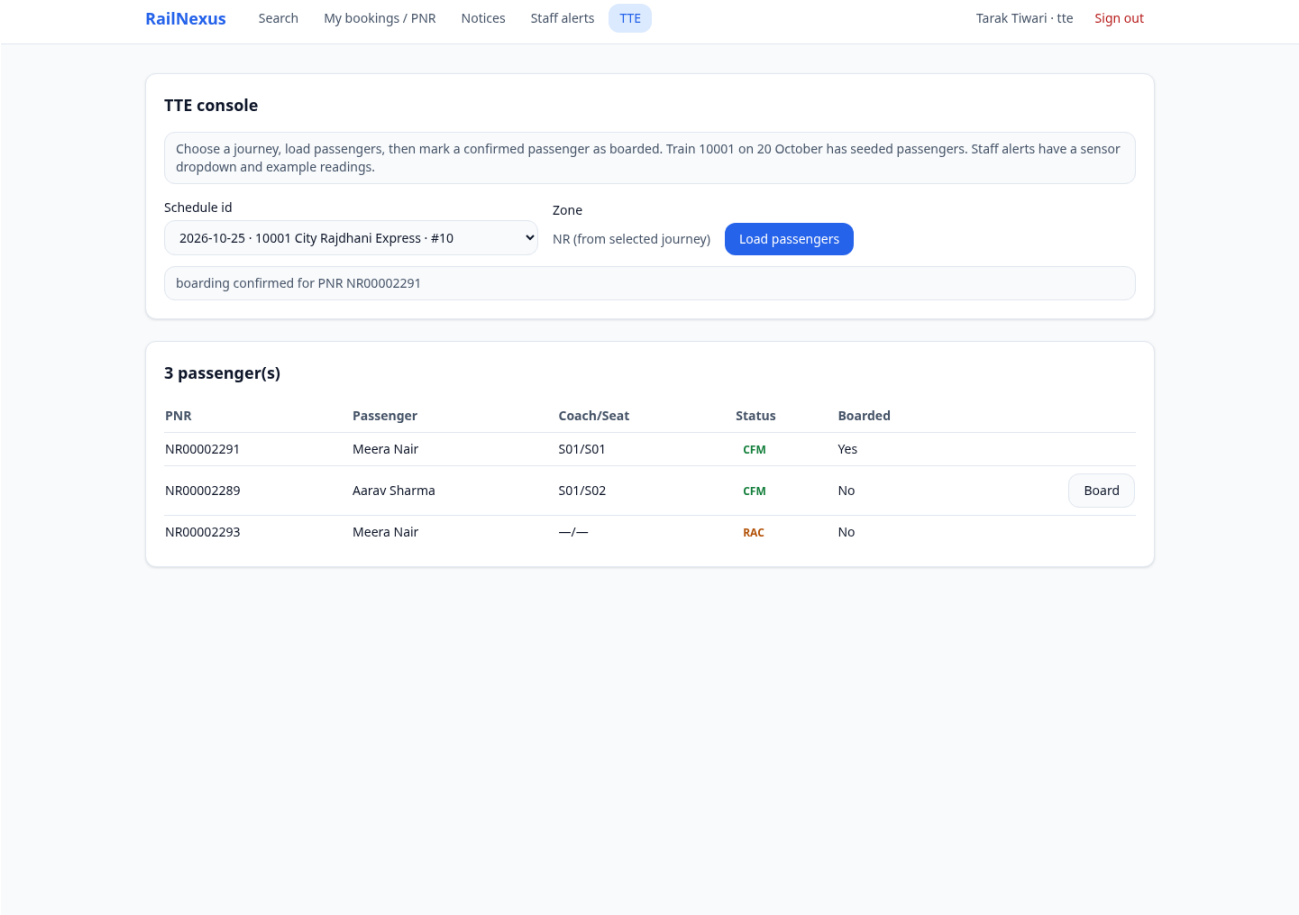


Figure 19.6. Actual RailNexus frontend screenshot, captured 23 October 2026.

19.7. Station Announcement Publication

The Station Master publishes an announcement for the assigned Riverside City station. The station selector is limited by staff scope, and the page displays successful publication.

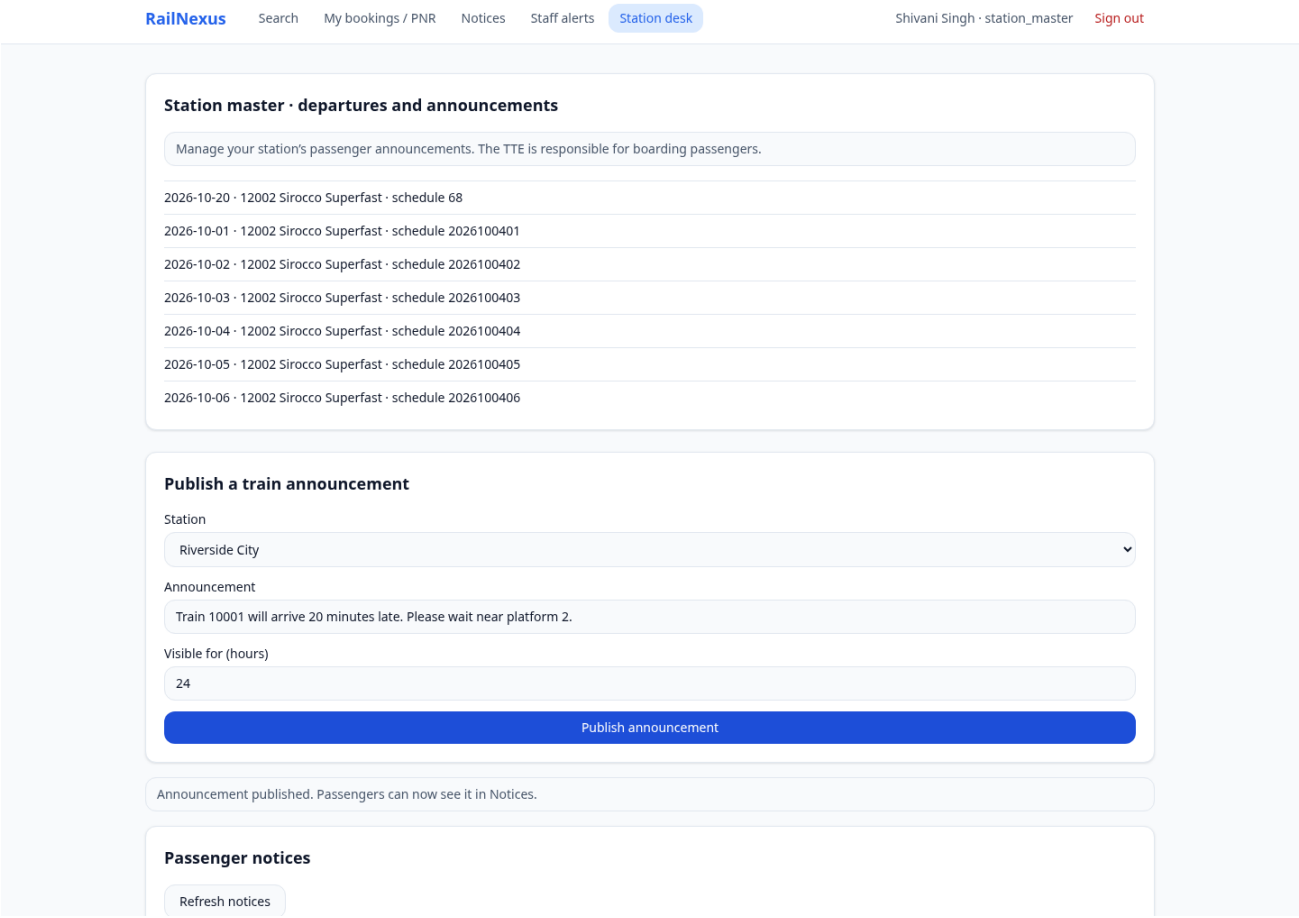


Figure 19.7. Actual RailNexus frontend screenshot, captured 23 October 2026.

19.8. Passenger Receives the Announcement

The Passenger Notices page shows the same message published by the Station Master: Train 10001 will arrive 20 minutes late. Please wait near platform 2. Publication and retrieval use the shared Oracle announcement records.

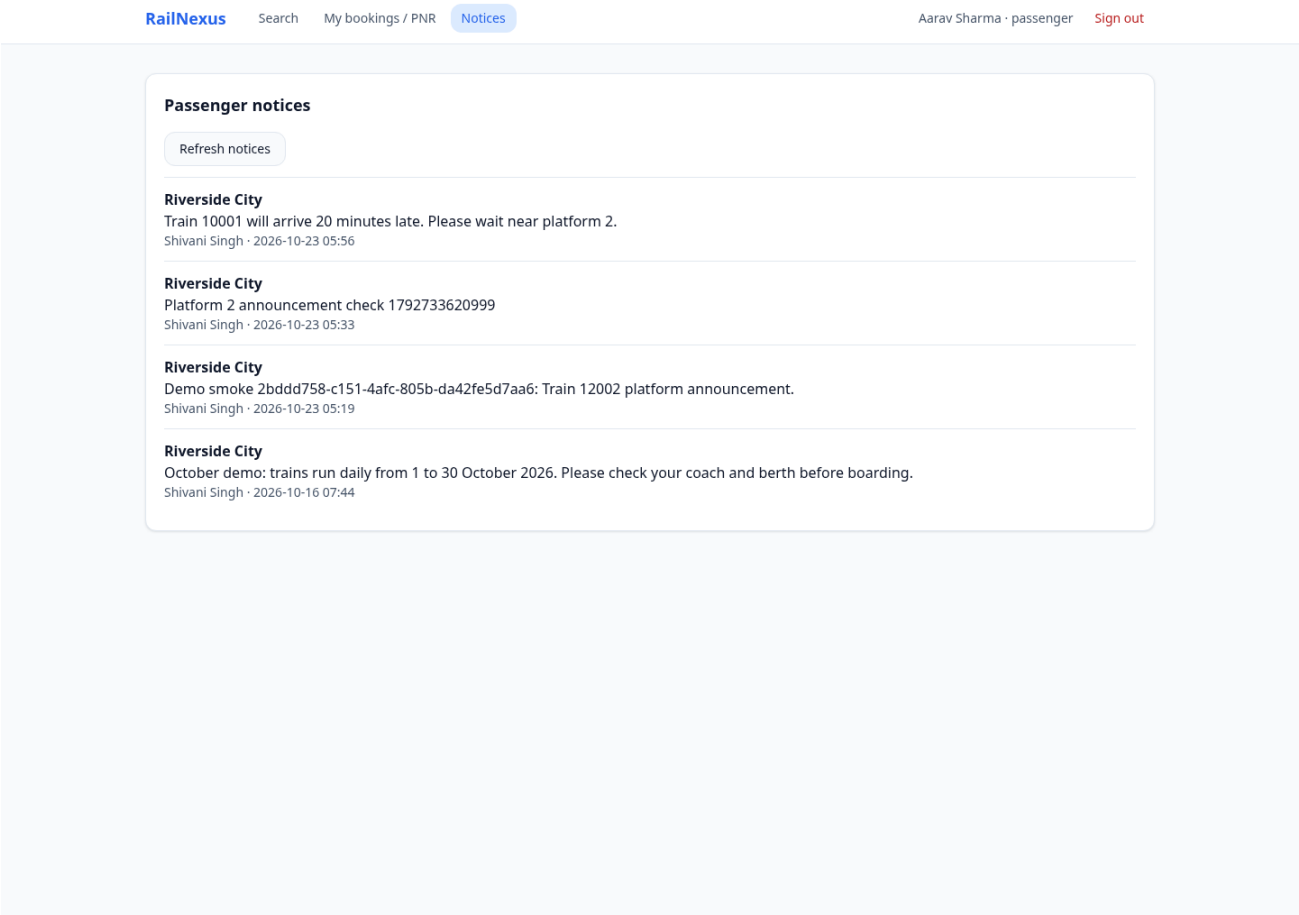


Figure 19.8. Actual RailNexus frontend screenshot, captured 23 October 2026.

19.9. Zone Administrator Reports

The Zone Administrator views reports restricted to the assigned Northern zone. Report values come from actual booking and payment data; they are not hard-coded dashboard totals. Cancelled history remains part of the booking count definition.

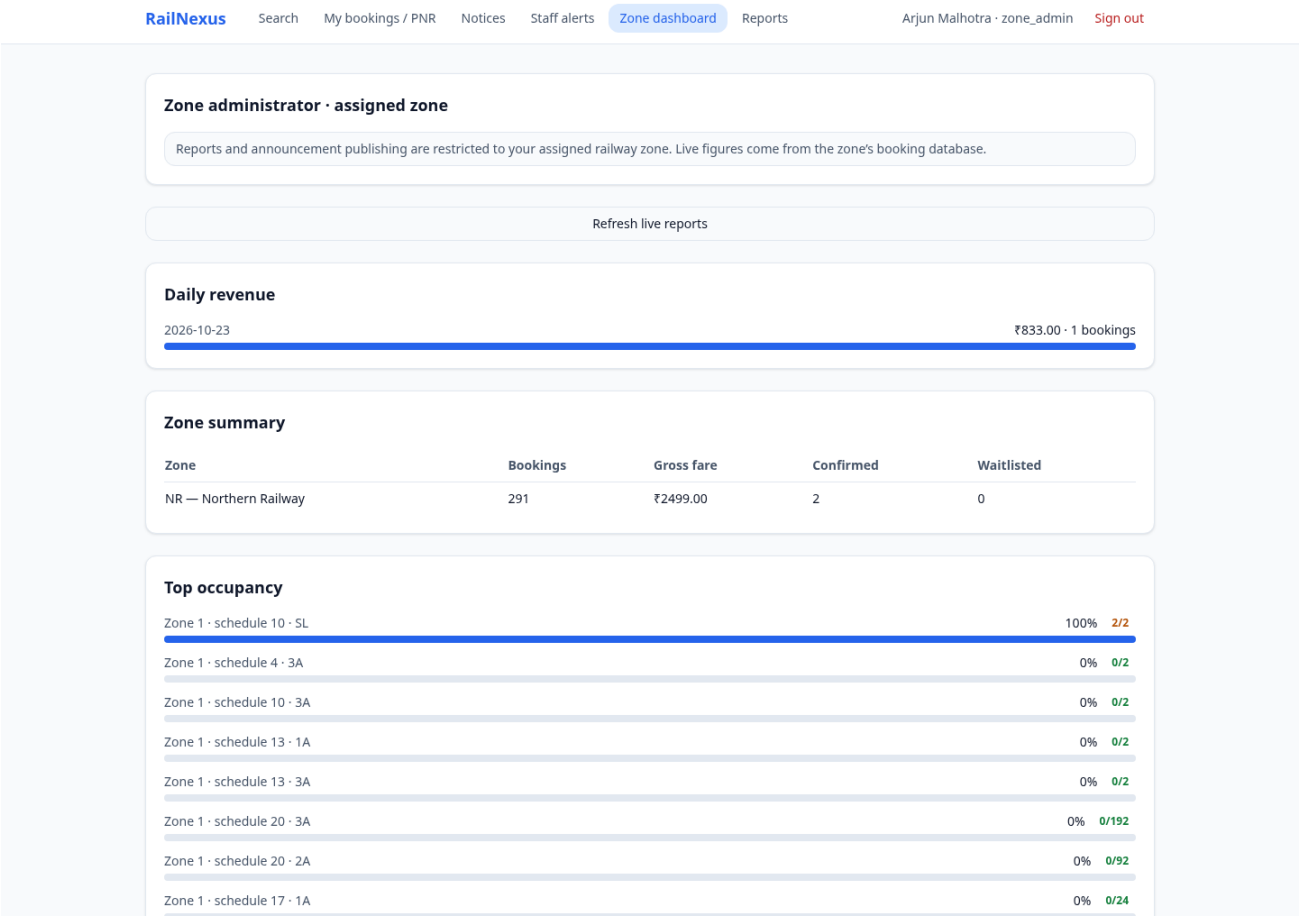


Figure 19.9. Actual RailNexus frontend screenshot, captured 23 October 2026.

19.10. Super Administrator Audit

The Super Administrator can inspect all-zone reporting and the Procedure and trigger activity table. Booking, cancellation and promotion events are recorded by the package/audit/outbox workflow.

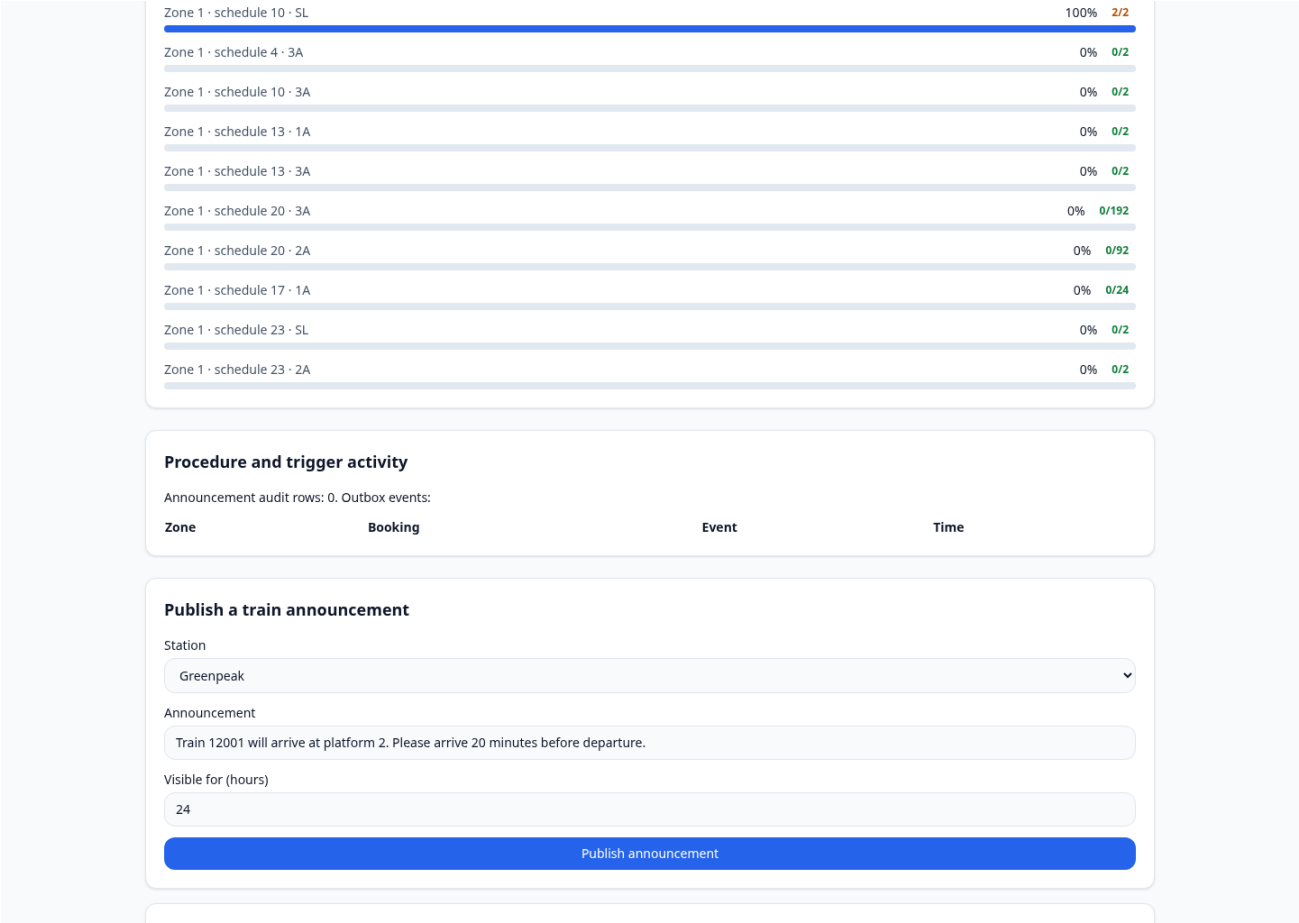


Figure 19.10. Actual RailNexus frontend screenshot, captured 23 October 2026.

20. Testing and Results

The limited-inventory implementation was checked through unit tests, real HTTP/Oracle requests and browser interaction. The recorded real API regression completed 276 assertions after the regular-train inventory change. Backend unit tests passed 31 tests and frontend unit tests passed two tests. Both TypeScript production builds completed successfully.

A normal-browser reservation test created four regular-train tickets with statuses CFM, CFM, RAC and WL. Cancelling a confirmed ticket changed the queued statuses to CFM and RAC. The test used Search, Booking and My bookings and released the test reservations afterwards.

A same-browser two-tab test verified the login-storage fix. Station Master published after Passenger signed in in the other tab; Passenger saw the exact message. Both sessions survived refresh, and an attempted Passenger publication remained forbidden. Screenshot capture repeated the linked two-passenger and staff workflows against the running application.

Check	Expected result	Recorded outcome
Six routes × 30 dates	At least one eligible train per route/day	180 searches passed
Four reservations in a fresh class	CFM, CFM, RAC, WL	Passed
Cancel confirmed ticket	RAC → CFM; WL → RAC	Passed
Captured payment cancellation	Refund uses captured amount	Passed
Ownership and role restrictions	Other passenger / forbidden role denied	Passed
Station Master → Passenger	Same announcement appears in Notices	Passed
TTE boarding	Confirmed passenger becomes boarded	Passed
Inventory audit	Two usable seats per October class in all zones	Zero mismatched classes

Evidence interpretation

The 276-assertion run is the recorded limited-inventory regression, not a claim that every optional ADBMS extension was independently exercised. Screenshot reservations were cancelled after capture to release the test inventory.

21. Limitations and Future Enhancements

The prototype is designed for a classroom demonstration. Payments are simulated and no railway-operator, banking or SMS gateway is integrated. The two-seat inventory is deliberate and should not be described as realistic train capacity.

Zone fragmentation is logical: the three zone schemas share one Oracle instance. The deployment does not demonstrate physical multi-node failover, replicated availability or measured production-scale throughput. The schedule-level lock also serializes conflicting booking decisions more broadly than a highly optimized production system might.

RAC is represented here as a queue awaiting a full berth. It does not implement every commercial railway rule concerning shared berths, quotas, boarding detachment or chart preparation. The payment lifecycle lacks an automatic expiration policy for unpaid confirmed reservations.

Passenger pages normally require refresh or reopening the ticket to see changes made by another session. MongoDB IoT snapshots are best-effort, and the optional outbox relay must be enabled and verified before claiming downstream event delivery. Supporting JSON and object-relational scripts are broader than the frontend walkthrough.

Future work includes expiring unpaid reservations, production identity management, improved monitoring, real payment-provider integration, richer RAC rules, physically separated zone deployment, automated chart preparation and independently measured concurrency/load tests.

22. Conclusion

RailNexus demonstrates an end-to-end reservation lifecycle with observable effects across passenger and staff accounts. The small regular-train inventory makes confirmed booking, RAC, waiting list, cancellation and promotion directly visible through the ordinary frontend pages.

The database design uses EER specialization for person categories, employee roles, train categories, tickets and payment details. Relational keys and constraints support integrity, while PL/SQL packages implement the important workflows. Booking-audit and announcement triggers provide automatic database actions without being confused with queue-selection logic.

Oracle stores authoritative transactional records. Redis supports caching and MongoDB supports sensor snapshots. Zone schemas and database-link reporting demonstrate logical fragmentation and federation. Views and an on-demand materialized view provide different reporting mechanisms.

The implemented project therefore connects database concepts to practical results: one passenger’s cancellation changes another passenger’s ticket; a Station Master’s publication appears in Passenger Notices; a booking appears in the TTE manifest; and administrator reports and audits reflect the resulting database activity. The report and presentation use actual application screenshots and explicitly record the prototype’s limits.

References

[1] RailwayReservationSystem source repository. https://github.com/priyaanshuu21/RailwayReservationSystem. Reviewed branch: demo/october-2026-actor-fixes; revision a941142; 23 October 2026.

[2] Oracle Database PL/SQL Language Reference. Oracle documentation. https://docs.oracle.com/en/database/oracle/oracle-database/23/lnpls/ . Reference for packages, procedures, functions, exceptions and triggers.

[3] Oracle Database SQL Language Reference. https://docs.oracle.com/en/database/oracle/oracle-database/23/sqlrf/ . Reference for constraints, views, materialized views and SQL locking clauses.

[4] node-oracledb documentation. https://node-oracledb.readthedocs.io/ . Reference for connection pools, binds and PL/SQL calls.

[5] Redis documentation. https://redis.io/docs/latest/ . Reference for caching and pub/sub concepts.

[6] MongoDB Manual: Time Series Collections. https://www.mongodb.com/docs/manual/core/timeseries-collections/ . Reference for sensor-oriented document storage.

[7] React documentation. https://react.dev/ . Reference for frontend components and state.

[8] Docker Compose documentation. https://docs.docker.com/compose/ . Reference for containerized local services.

Source basis

Implementation claims were checked against the repository and observed application behaviour. External references provide conceptual background; they are not substituted for project evidence.

Appendix A. Concept-to-File Map

The following map allows the implementation of each concept to be located during review. These are repository-relative file paths.

Concept	Primary source location
Person / employee / train EER, subtype triggers	db/shared_ddl/01_core_schema.sql
Booking / seat / ticket / payment schema	db/zone_common/booking_family.sql
Booking, RAC, WL, cancellation, promotion	db/plsql/pkg_booking.sql
Fare calculation function	db/plsql/pkg_fare.sql
Payment / refund procedures	db/plsql/pkg_payment.sql
TTE boarding procedure	db/plsql/pkg_tte.sql
Announcement procedure and audit trigger	db/plsql/pkg_announcements.sql
Booking audit → outbox trigger	db/plsql/outbox_audit_trigger.sql
Zone database links and federation	db/distributed/db_links.sql; distributed_views.sql
Views and materialized view	db/reporting/report_views.sql
Object-relational types and methods	db/or/route_journey_types.sql; pkg_journey.sql
JSON rule functions	db/json/pkg_json_rules.sql
Sensor processing and Mongo snapshots	db/plsql/pkg_iot.sql; backend/src/iot/iotService.ts
Station cache / role checks	backend/src/cache; backend/src/api/middleware/auth.ts
Limited inventory migration / setup	db/demo/limited_inventory.sql; scripts/prepare-demo.mjs
Real API regression	scripts/demo-smoke.mjs